

Module 22 Structured Notes: Choosing the Right Structure

1. Current Progress Analysis

You have already studied almost the full linked-list path.

Before Module 22, you learned how to build and operate on linked lists.

Area	What You Learned
Arrays vs linked lists	Why linked lists exist and how arrays differ from them
Node and pointer basics	How nodes store data and links
Traversal	How to move through a list using pointers
Recursion	How recursive functions work on linked lists
Counting, summing, max, min	How to process every node in a list
Searching	How to find a value in a linked list
Insertion	How to add nodes by changing links
Deletion	How to remove nodes safely
Reversal	How to reverse a linked list
Multi-list operations	How to concatenate and merge lists
Loop detection	How to detect whether a list has a cycle
C++ class implementation	How to organize linked-list code using classes
Circular singly linked lists	How a singly linked list can loop back to the start
Doubly linked lists	How nodes can move forward and backward
Circular doubly linked lists	How to combine circular structure with two-way movement

So your current level is this:

Area	Status
Basic linked-list mechanics	Covered
Singly linked-list operations	Covered
Circular singly linked lists	Covered

Area	Status
Doubly linked lists	Covered
Circular doubly linked lists	Covered
Data-structure selection	Current topic

The important shift is:

Modules 1–21 taught you how to build structures.
Module 22 teaches you when to choose each structure.

Module 22 is a decision-making module.

It helps you answer this question:

Given a problem, which structure should I use, and why?

2. Big Idea of Module 22

Module 22 is not mainly about writing new pointer code.

It is about choosing the correct data structure.

A beginner may think:

Linked lists are better than arrays.

That is not true.

A better rule is:

Arrays are better for fast index access.
Linked lists are better for flexible growth and pointer-based insertion or deletion.

No structure is always best.

The best structure depends on what the program needs to do.

You should ask questions like:

1. Do I need fast access by index?
2. Do I know the size in advance?
3. Will I insert and delete often?

4. Do I need to move only forward?
5. Do I need to move backward too?
6. Should the structure stop at the end or loop back to the beginning?
7. How much extra memory can I use?

3. First Big Choice: Array or Linked List?

Before choosing a specific linked-list type, first decide whether you need an array or a linked list.

3.1 What Is an Array?

An array stores elements side by side in memory.

Example:

```
int A[5] = {10, 20, 30, 40, 50};
```

Memory picture:

Index:	0	1	2	3	4
Value:	10	20	30	40	50

The elements are stored continuously.

This is called contiguous memory.

Because the values are side by side, the computer can jump directly to an index.

Example:

```
A[3]
```

This directly accesses the value at index **3**.

So array access by index is:

```
O(1)
```

That means direct access takes constant time.

It does not matter whether the array has 5 elements or 5000 elements. If the index is known, access is still direct.

3.2 When Should You Use an Array?

Use an array when:

```
You know the size.  
You need fast index access.  
You often use A[i].  
You want binary search on sorted data.  
You do not insert/delete often at the beginning or middle.
```

Example:

Suppose you have marks for exactly 100 students.

You often need to access marks using roll number.

An array is a good choice because:

```
The size is known.  
Index access is fast.
```

3.3 Weaknesses of Arrays

Arrays are not always good.

Their main weakness is fixed size.

Example:

```
int A[5];
```

This array has exactly 5 spaces.

If you need 6 spaces, the array is too small.

If you create 1000 spaces but use only 30, memory is wasted.

Another weakness is insertion and deletion.

If you insert at the beginning or middle, many values may need to shift.

Example:

```
Original array:
```

```
10 20 30 40
```

```
Insert 5 at the beginning:
```

```
5 10 20 30 40
```

To make space for `5`, the other values must move right.

That shifting can take:

```
O(n)
```

3.4 What Is a Linked List?

A linked list stores values in separate blocks called nodes.

Each node usually stores:

```
data + link
```

Example:

```
10 -> 20 -> 30 -> NULL
```

Each node points to the next node.

A linked list does not need one large continuous block of memory.

The nodes can be stored in different places in heap memory.

The order is controlled by pointers, not by physical position in memory.

3.5 Linked List Access by Index

A linked list cannot jump directly to an index.

Suppose we want the fourth node.

We cannot directly write something like:

```
list[3]
```

In a normal linked list, we must move step by step:

```
first -> second -> third -> fourth
```

So linked-list access by index is:

```
O(n)
```

This is slower than array index access.

3.6 When Should You Use a Linked List?

Use a linked list when:

```
The size changes often.  
You do not know the final size in advance.  
You insert/delete often near the beginning.  
You want to avoid shifting many values.  
The records are large and moving them would be expensive.
```

Example:

Suppose users keep adding comments.

You do not know how many comments there will be.

A linked list can create one node at a time.

That makes linked lists useful for flexible growth.

4. Array vs Linked List Comparison

Feature / Operation	Array	Linked List
Memory layout	Contiguous, side by side	Separate nodes connected by pointers
Size	Fixed after creation	Can grow and shrink
Access by index	$O(1)$	$O(n)$
Linear search	$O(n)$	$O(n)$
Binary search	Good if sorted	Bad choice
Insert/delete at beginning	$O(n)$, because shifting is needed	Often $O(1)$, by changing head links
Insert/delete at known node	Not pointer-based	Can be efficient by changing links
Extra memory	No pointer field per element	Extra pointer fields needed
Best use case	Fast lookup by index	Dynamic insertion/deletion

Simple rule:

Need $A[i]$ often? Use an array.
Need flexible size and frequent insertion/deletion? Use a linked list.

5. Why Binary Search Is Good for Arrays but Bad for Linked Lists

Binary search works by checking the middle element repeatedly.

For an array:

[3, 7, 10, 15, 20, 30, 40]

The computer can directly access the middle using an index.

Example:

A[mid]

Array middle access is:

$O(1)$

So binary search on a sorted array is efficient:

$O(\log n)$

But in a linked list, there is no direct middle access.

To reach the middle node, we must start from the first node and walk step by step.

That removes the main advantage of binary search.

Important conclusion:

Binary search is good for sorted arrays.
Binary search is not a good choice for normal linked lists.

6. Second Big Choice: Which Linked List?

After deciding that you need a linked list, the next question is:

Which type of linked list should I use?

You have learned four main types:

1. Linear singly linked list
2. Circular singly linked list
3. Linear doubly linked list
4. Circular doubly linked list

Each type has different strengths and weaknesses.

7. Linear Singly Linked List

7.1 Shape

A linear singly linked list looks like this:

```
10 -> 20 -> 30 -> NULL
```

It has a clear end.

The last node points to:

```
NULL
```

7.2 Node Structure

A singly linked-list node stores:

```
data + next
```

C structure:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

The `next` pointer stores the address of the next node.

7.3 Movement

Movement is forward only.

```
p = p->next;
```

A singly linked list cannot move backward directly.

If you are at node 30, you cannot directly go back to node 20.

7.4 Strengths

Linear singly linked lists are:

Simple.
Easy to understand.
Memory-efficient compared with doubly linked lists.
Good for forward-only movement.
Good for head insertion and head deletion.

Head insertion can be done with:

```
t->next = first;  
first = t;
```

This is usually:

$O(1)$

7.5 Weaknesses

Linear singly linked lists have these weaknesses:

They cannot move backward.
They do not support direct index access.
Deleting a known node may still require finding the previous node.
Tail operations may require traversal unless a last pointer is stored.

Example:

10 -> 20 -> 30 -> NULL

If you want to delete 30, you usually need the node before it, which is 20.

In a singly linked list, node 30 does not know who comes before it.

So you may need to traverse from the beginning.

7.6 When to Use a Linear Singly Linked List

Use a linear singly linked list when:

You only need forward movement.
You want the simplest linked-list structure.
Memory matters.
You do not need to move backward.

Example uses:

Simple task list processed from front to back.
Stack-like structure.
Simple chain of records.

8. Circular Singly Linked List

8.1 Shape

A circular singly linked list looks like this:

```
10 -> 20 -> 30
^           |
|           |
|_____||
```

The last node does not point to `NULL`.

Instead, the last node points back to the first node.

If the first node is called `Head`, then:

```
last->next = Head;
```

8.2 Node Structure

A circular singly linked list uses the same node structure as a normal singly linked list:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

The node structure is the same.

The difference is the final link.

Linear singly linked list:

```
last->next = NULL
```

Circular singly linked list:

```
last->next = Head
```

8.3 Movement

Movement is forward, then it loops back to `Head`.

```
10 -> 20 -> 30 -> 10 -> 20 -> 30 -> ...
```

This is useful when the process repeats.

8.4 Important Traversal Warning

For a normal linear list, traversal often uses:

```
while (p != NULL)
```

But this is wrong for a circular linked list.

Why?

Because a circular list usually never reaches `NULL`.

If you use:

```
while (p != NULL)
```

on a circular list, the loop may run forever.

Correct idea:

```
Start at Head.  
Visit nodes.  
Stop when you return to Head.
```

That is why circular list display usually uses a `do-while` loop.

8.5 Strengths

Circular singly linked lists are useful because:

```
They support repeated cyclic movement.  
They do not need NULL at the end.  
They are good for round-robin style problems.  
They can keep rotating through nodes.
```

8.6 Weaknesses

Circular singly linked lists have these weaknesses:

```
They can loop forever if traversal is written incorrectly.  
They cannot move backward.  
Head insertion/deletion may require finding the last node if no last pointer is  
stored.
```

8.7 Head Insertion Complexity Warning

A common beginner mistake is thinking:

Circular singly head insertion is always $O(1)$.

That is not always true.

In a circular singly linked list, the last node must point to **Head**.

If a new node becomes the new **Head**, then the old last node must point to the new **Head**.

That means we may need to find the last node first.

Example:

```
Head
|
v
10 -> 20 -> 30
^           |
|           |
```

Insert **5** before **10**.

Final result:

```
Head
|
v
5 -> 10 -> 20 -> 30
^           |
|           |
```

Now **30->next** must point to **5**.

If we do not already have a **last** pointer, we must traverse to **30**.

That traversal costs:

$O(n)$

So:

Circular singly head insertion is $O(n)$ if no last pointer is stored.
Circular singly head insertion can be $O(1)$ if a last pointer is maintained.

8.8 When to Use a Circular Singly Linked List

Use a circular singly linked list when:

You need cyclic forward movement.
You do not need backward movement.
You want the last node to naturally return to the first node.

Example uses:

Round-robin scheduling.
Circular turn system.
Repeated playlist traversal in one direction.

9. Linear Doubly Linked List

9.1 Shape

A linear doubly linked list looks like this:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

The first node has no previous node.

So:

```
first->prev = NULL
```

The last node has no next node.

So:

```
last->next = NULL
```

9.2 Node Structure

A doubly linked-list node stores:

```
prev + data + next
```

C structure:

```
struct Node {  
    struct Node *prev;  
    int data;  
    struct Node *next;  
};
```

Meaning:

Field	Meaning
prev	Address of the previous node
data	Value stored in the node
next	Address of the next node

9.3 Movement

A doubly linked list can move forward and backward.

Forward movement:

```
p = p->next;
```

Backward movement:

```
p = p->prev;
```


Example:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

From node `20`:

```
20->prev points to 10  
20->next points to 30
```

9.4 Strengths

Linear doubly linked lists are useful because:

```
They can move in both directions.  
They make backward navigation easy.  
Deleting a node is easier if you already have a pointer to that node.  
They are good for navigation-style problems.
```

9.5 Why Deletion Can Be Easier in a Doubly Linked List

In a singly linked list, a node only knows the next node.

Example:

```
10 -> 20 -> 30 -> NULL
```

If you want to delete `20`, you need the previous node `10` so you can write:

```
previous->next = target->next;
```

But in a doubly linked list, the target node already has `prev`.

Example:

```
10 <-> 20 <-> 30
```

If `p` points to `20`, then:

```
p->prev points to 10  
p->next points to 30
```

So we can connect the neighbors:

```
p->prev->next = p->next;  
p->next->prev = p->prev;
```

This is why doubly linked lists are often better when deletion is frequent and you already have a pointer to the node.

9.6 Weaknesses

Linear doubly linked lists have these weaknesses:

```
They use more memory than singly linked lists.  
Each node has two pointer fields instead of one.  
Insertion and deletion require more link updates.  
The code is more complex than singly linked-list code.
```

9.7 When to Use a Linear Doubly Linked List

Use a linear doubly linked list when:

```
You need forward and backward movement.  
You frequently move previous and next.  
You frequently delete nodes when you already have the node pointer.  
You are building a navigation-style structure.
```

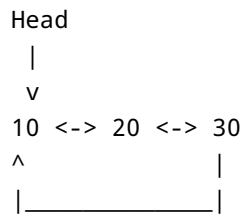
Example uses:

```
Browser history.  
Undo and redo navigation.  
Music playlist with previous and next.  
Text editor cursor movement.
```

10. Circular Doubly Linked List

10.1 Shape

A circular doubly linked list looks like this:



It combines two ideas:

Circular structure + doubly linked movement

That means:

The list can move forward.
The list can move backward.
The last node connects back to Head.
Head connects backward to the last node.

10.2 Node Structure

A circular doubly linked list uses the same node structure as a normal doubly linked list:

```
struct Node {  
    struct Node *prev;  
    int data;  
    struct Node *next;  
};
```

The structure is the same.

The difference is the links at the ends.

Linear doubly linked list:

```
first->prev = NULL  
last->next = NULL
```

Circular doubly linked list:

```
Head->prev = last  
last->next = Head
```

10.3 Important Invariants

An invariant is a rule that must stay true while the structure is correct.

For a circular doubly linked list, these rules must stay true:

```
last->next == Head  
Head->prev == last
```

Also, neighboring links must agree.

If node **A** comes before node **B**, then:

```
A->next == B  
B->prev == A
```

For a one-node circular doubly linked list:

```
Head->next == Head  
Head->prev == Head
```

The node points to itself in both directions.

10.4 Movement

Forward movement:

```
p = p->next;
```

Backward movement:

```
p = p->prev;
```

Because the list is circular, movement can continue forever if there is no proper stopping condition.

Example forward movement:

```
10 -> 20 -> 30 -> 10 -> 20 -> 30 -> ...
```

Example backward movement:

```
10 -> 30 -> 20 -> 10 -> 30 -> 20 -> ...
```

10.5 Strengths

Circular doubly linked lists are powerful because:

```
They can move forward.  
They can move backward.  
They can loop around.  
They can reach the last node quickly using Head->prev.  
They are flexible for insertion and deletion near Head.
```

This is the most flexible linked-list type covered in this course.

10.6 Weaknesses

Circular doubly linked lists also have costs:

```
They use the most pointer memory.  
They are the most complex to implement correctly.  
Every insertion must preserve prev, next, and circular links.
```

Every deletion must preserve prev, next, and circular links.
Wrong link updates can break the whole structure.

So they are powerful, but not always necessary.

Do not use a circular doubly linked list just because it has many features.

Use it only when those features are needed.

10.7 When to Use a Circular Doubly Linked List

Use a circular doubly linked list when:

You need circular movement.
You need backward movement.
You want quick access to the last node from Head.
You need next and previous behavior in a loop.

Example uses:

Circular playlist with next and previous.
Turn-based game where players can move in both directions.
Deque-like circular navigation.
Rotating menu with forward and backward movement.

11. Comparison of Linked-List Types

Structure	Node Links	Ends at NULL?	Direction	Head Insert/Delete
Linear singly linked list	<code>next</code>	Yes	Forward only	O(1)
Circular singly linked list	<code>next</code>	No, returns to Head	Forward loop	O(n) if no last pointer
Linear doubly linked list	<code>prev</code> , <code>next</code>	Yes	Forward and backward	O(1)
Circular doubly linked list	<code>prev</code> , <code>next</code>	No, returns to Head	Forward and backward loop	O(1) using <code>Head->prev</code>

12. Memory Cost Comparison

Each structure uses a different amount of memory per node.

12.1 Array

An array stores only values.

Example:

```
[10] [20] [30]
```

There is no pointer field inside each element.

So arrays usually have lower per-element memory cost.

12.2 Singly Linked List

A singly linked-list node stores:

```
data + next
```

So each node needs one pointer field.

12.3 Doubly Linked List

A doubly linked-list node stores:

```
prev + data + next
```

So each node needs two pointer fields.

This uses more memory than a singly linked list.

12.4 Circular Lists

Circular lists do not necessarily add extra fields.

A circular singly linked list still has:

data + next

A circular doubly linked list still has:

prev + data + next

The difference is how the links are arranged.

Circular structure changes the final links, not the node fields.

13. Complexity Summary

Operation / Need	Array	Linear Singly	Circular Singly	Linear Doubly	Circular Doubly
Access by index	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Linear search	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Binary search	Good if sorted	Poor	Poor	Poor	Poor
Head insertion	$O(n)$ shifting	$O(1)$	$O(n)$ without last pointer	$O(1)$	$O(1)$
Head deletion	$O(n)$ shifting	$O(1)$	$O(n)$ without last pointer	$O(1)$	$O(1)$
Move forward	Yes	Yes	Yes	Yes	Yes
Move backward	By index	No	No	Yes	Yes
Circular movement	No	No	Yes	No	Yes
Extra pointer memory	No	One pointer	One pointer	Two pointers	Two pointers

Important note:

Complexity depends on what pointer you already have.

For example:

Inserting after a known node in a linked list can be $O(1)$.
Finding that node first may take $O(n)$.

So always separate these two questions:

1. How long does it take to find the position?
2. How long does it take to update the links after the position is found?

14. Decision Rules

Rule 1: Need Fast Index Access?

Use an array.

Example:

Get the 500th score quickly.

Best choice:

Array

Reason:

$A[499]$ is $O(1)$.

A linked list would need to walk through nodes one by one.

Rule 2: Need Binary Search?

Use a sorted array.

Example:

Sorted student IDs with frequent lookup.

Best choice:

Sorted array

Reason:

Binary search needs fast middle access.

Arrays provide fast middle access.

Linked lists do not.

Rule 3: Unknown Size and Frequent Growth?

Use a linked list.

Example:

Users keep adding comments, and we do not know how many comments there will be.

Best choice:

Linked list

Reason:

Nodes can be created one by one.

Rule 4: Only Forward Movement Needed?

Use a singly linked list.

Example:

Process tasks from start to end.

Best choice:

Linear singly linked list

Reason:

No need to pay memory cost for prev.

Rule 5: Need Previous and Next?

Use a doubly linked list.

Example:

Browser back and forward buttons.

Best choice:

Doubly linked list

Reason:

next moves forward.
prev moves backward.

Rule 6: Need Repeated Cycle?

Use a circular linked list.

Example:

Players take turns: A -> B -> C -> A -> B -> C ...

Best choice:

Circular linked list

Reason:

After the last player, movement returns to the first player.

If movement is only forward, circular singly linked list may be enough.

Rule 7: Need Cycle Plus Backward Movement?

Use a circular doubly linked list.

Example:

A circular playlist where the user can press next and previous.

Best choice:

Circular doubly linked list

Reason:

next gives the next song.
prev gives the previous song.
last->next returns to the first song.
Head->prev gives the last song.

15. Scenario Practice

Scenario A: Fixed-Size Exam Marks

Requirement:

There are 100 students.
Marks are accessed by roll number often.

Best choice:

Array

Reason:

The size is known.
Index access is fast.

Scenario B: Browser History

Requirement:

Move backward and forward between pages.

Best choice:

Doubly linked list

Reason:

Back uses prev.
Forward uses next.

Scenario C: Round-Robin CPU Scheduling

Requirement:

Each process gets a turn, then the system cycles back to the first process.

Best choice:

Circular singly linked list

Reason:

Movement is naturally circular and usually forward only.

Scenario D: Music Playlist with Next, Previous, and Looping

Requirement:

Move next.
Move previous.
Loop from last to first.
Loop from first to last when moving backward.

Best choice:

Circular doubly linked list

Reason:

It supports forward, backward, and circular movement.

Scenario E: Sorted Data with Frequent Binary Search

Requirement:

Find values quickly in sorted data.

Best choice:

Array

Reason:

Binary search is efficient because array middle access is $O(1)$.

Scenario F: Unknown Number of Large Records

Requirement:

The number of records is unknown.
Records are large.
There are frequent insertions and deletions near the beginning.

Best choice:

Linked list

Reason:

A linked list avoids shifting large records.
Nodes can be created and removed one at a time.

Scenario G: Simple Forward Task Processing

Requirement:

Tasks are processed from first to last.
No backward movement is needed.

Best choice:

Linear singly linked list

Reason:

It is simple and uses less memory than a doubly linked list.

Scenario H: Circular Game Turns with Reverse Direction

Requirement:

Players sit in a circle.
The game can move clockwise or counterclockwise.

Best choice:

Circular doubly linked list

Reason:

Circular structure handles the loop.
Doubly linked structure handles both directions.

16. Common Beginner Mistakes

Mistake 1: Thinking Linked Lists Are Always Better

Wrong idea:

Linked lists are better than arrays.

Correct idea:

Arrays are better for direct index access.
Linked lists are better for flexible insertion and deletion.

Mistake 2: Using Binary Search on a Linked List

Wrong idea:

The linked list is sorted, so binary search will be efficient.

Correct idea:

Binary search needs fast middle access.
Linked lists cannot jump to the middle in $O(1)$.

Mistake 3: Using `while (p != NULL)` on a Circular List

Wrong code idea:

```
while (p != NULL) {  
    p = p->next;  
}
```

Why it is wrong:

A circular list may never reach NULL.

Correct idea:

Stop when p returns to Head.

Mistake 4: Forgetting the `prev` Link in a Doubly Linked List

Wrong idea:

Only update next links during insertion or deletion.

Correct idea:

A doubly linked list must keep both next and prev links correct.

If forward traversal works but backward traversal is broken, the doubly linked list is still incorrect.

Mistake 5: Thinking Circular Singly Head Insertion Is Always $O(1)$

Correct idea:

It is $O(n)$ if you must find the last node.
It is $O(1)$ if you already maintain a last pointer.

Mistake 6: Choosing the Most Complex Structure Without Need

Wrong idea:

Circular doubly linked list has the most features, so it is always best.

Correct idea:

More features also mean more memory and more complex code.
Use only the features the problem needs.

17. Final Beginner Summary

Use this mental model:

Array = fast jumping by index.
Singly linked list = simple forward chain.
Circular singly linked list = forward chain that loops.
Doubly linked list = forward and backward chain.
Circular doubly linked list = forward, backward, and looping chain.

The correct structure depends on the operations.

There is no universal best structure.

Choose based on:

Access pattern.
Insertion pattern.
Deletion pattern.
Traversal direction.
Memory cost.

Whether the structure is linear or circular.
Whether the program needs forward-only or two-way movement.

18. One-Page Decision Guide

Choose an Array If:

You know the size.
You need fast access by index.
You need binary search.
You rarely insert/delete in the middle or beginning.

Choose a Linear Singly Linked List If:

You need a simple dynamic list.
You only move forward.
You want low pointer memory cost.
You often insert/delete at the head.

Choose a Circular Singly Linked List If:

You need forward cyclic movement.
The last item should return to the first item.
Backward movement is not needed.

Choose a Linear Doubly Linked List If:

You need forward and backward movement.
You need previous/next navigation.
You frequently delete known nodes.

Choose a Circular Doubly Linked List If:

You need circular movement.
You need both next and previous movement.
You want quick access from Head to the last node using Head->prev.

19. Mastery Check

You understand Module 22 if you can answer these questions:

1. Why is binary search good for arrays but bad for linked lists?
2. Why does a doubly linked list use more memory than a singly linked list?
3. Why is circular singly head insertion $O(n)$ if no last pointer is stored?
4. When would you choose a circular doubly linked list even though it uses more memory?
5. Why is there no single best data structure?
6. When is an array better than a linked list?
7. When is a singly linked list better than a doubly linked list?
8. Why is `while (p != NULL)` dangerous in circular lists?
9. What does `Head->prev` give you in a circular doubly linked list?
10. What is the main cost of adding more pointer links?

20. Key Answer of Module 22

The main lesson of Module 22 is:

Choose the data structure based on the problem.

More specifically:

Choose based on access pattern, insertion/deletion pattern, traversal direction, memory cost, and whether the structure must be linear or circular.

A good programmer does not ask:

Which structure is always best?

A good programmer asks:

What operations will this program perform most often?

Then the correct structure becomes much easier to choose.